

My Project

Создано системой Doxygen 1.9.8

Глава 1

Алфавитный указатель классов

1.1 Классы

Классы с их кратким описанием.

Connection	Класс для управления сетевыми соединениями и аутентификацией	??
Params	Структура для хранения параметров командной строки	??
UserInterface	Класс для обработки параметров командной строки	??

Глава 2

Список файлов

2.1 Файлы

Полный список документированных файлов.

connection.cpp	Реализация сетевого соединения и аутентификации	??
connection.h	Заголовочный файл для класса Connection - установка сетевых соединений	??
crypto.cpp	Реализация криптографических функций	??
crypto.h	Заголовочный файл для криптографических функций	??
interface.cpp	Реализация пользовательского интерфейса	??
interface.h	Заголовочный файл для пользовательского интерфейса	??
log.cpp	Реализация функций логирования	??
log.h	Заголовочный файл для функций логирования	??
main.cpp	Главный модуль серверного приложения	??

Глава 3

Классы

3.1 Класс Connection

Класс для управления сетевыми соединениями и аутентификацией

```
#include <connection.h>
```

Открытые статические члены

- static int `establishConnection` (const `Params` *p)
Установка соединения с клиентом

3.1.1 Подробное описание

Класс для управления сетевыми соединениями и аутентификацией

Обеспечивает установку TCP-соединения, аутентификацию клиентов и обработку передаваемых данных

3.1.2 Методы

3.1.2.1 `establishConnection()`

```
int Connection::establishConnection (  
    const Params * connection_params ) [static]
```

Установка соединения с клиентом

Установка соединения и обработка клиентских запросов

Аргументы

in	p	Параметры соединения (адрес, порт, файлы конфигурации)
----	---	--

Возвращает

0 при успешном выполнении, 1 при ошибке аутентификации

Исключения

	std::system_error	при ошибках сетевого взаимодействия
	std::runtime_error	при некорректных данных

Аргументы

	in	connection_params	Параметры соединения
--	----	-------------------	----------------------

Возвращает

0 при успехе, 1 при ошибке аутентификации

Исключения

	std::system_error	при сетевых ошибках
	std::runtime_error	при некорректных данных

Выполняет полный цикл работы сервера: создание сокета, привязку, прослушивание, аутентификацию клиента и обработку данных

Объявления и описания членов классов находятся в файлах:

- [connection.h](#)
- [connection.cpp](#)

3.2 Структура Params

Структура для хранения параметров командной строки

```
#include <interface.h>
```

Открытые атрибуты

- string inFileName
Имя файла с учетными данными
- string inFileJournal
Имя файла журнала
- string inFileData
Имя файла с данными
- string logFile
Имя файла лога
- int Port
Порт для подключения
- string Address
Адрес для подключения

3.2.1 Подробное описание

Структура для хранения параметров командной строки

Объявления и описания членов структуры находятся в файле:

- [interface.h](#)

3.3 Класс `UIInterface`

Класс для обработки параметров командной строки

```
#include <interface.h>
```

Открытые члены

- [UIInterface](#) ()
Конструктор инициализирует опции командной строки
- `bool` [Parser](#) (int argc, const char **argv)
Парсинг аргументов командной строки
- `string` [getDescription](#) ()
Получение описания параметров
- [Params](#) [getParams](#) ()
Получение параметров приложения

3.3.1 Подробное описание

Класс для обработки параметров командной строки

Использует `boost::program_options` для парсинга аргументов

3.3.2 Конструктор(ы)

3.3.2.1 `UIInterface()`

```
UIInterface::UIInterface ( )
```

Конструктор инициализирует опции командной строки

Конструктор инициализирует параметры командной строки

3.3.3 Методы

3.3.3.1 getDescription()

```
std::string UserInterface::getDescription ( )
```

Получение описания параметров

Генерация строки с описанием параметров

Возвращает

Строка с описанием всех параметров

Форматированная строка с описанием всех опций

3.3.3.2 getParams()

```
Params UserInterface::getParams ( ) [inline]
```

Получение параметров приложения

Возвращает

Структура с параметрами

3.3.3.3 Parser()

```
bool UserInterface::Parser (
    int argc,
    const char ** argv )
```

Парсинг аргументов командной строки

Аргументы

	in	argc	Количество аргументов
	in	argv	Массив аргументов

Возвращает

true при успешном парсинге, false при запросе помощи или ошибке

Аргументы

	in	argc	Количество аргументов
	in	argv	Массив аргументов

Возвращает

`true` при успешном парсинге, `false` при запросе помощи

Исключения

<code>boost::program_options::error</code>	при некорректных аргументах
--	-----------------------------

Объявления и описания членов классов находятся в файлах:

- [interface.h](#)
- [interface.cpp](#)

Глава 4

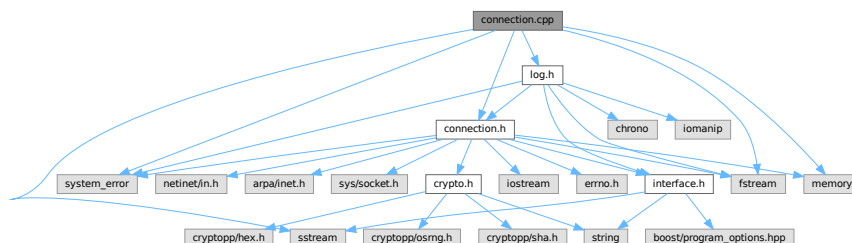
Файлы

4.1 Файл connection.cpp

Реализация сетевого соединения и аутентификации

```
#include "connection.h"  
#include "log.h"  
#include <fstream>  
#include <sstream>  
#include <memory>  
#include <system_error>
```

Граф включаемых заголовочных файлов для connection.cpp:



Функции

- `bool findUserInCredentialsFile (const std::string &filename, const std::string &username, std::string &password)`
Поиск пользователя в файле учетных записей

4.1.1 Подробное описание

Реализация сетевого соединения и аутентификации

Автор

Мезин А.А.

Версия

1.0

Дата

2025

Авторство

ИБСТ ПГУ

Предупреждения

Учебный пример

4.1.2 Функции

4.1.2.1 findUserInCredentialsFile()

```
bool findUserInCredentialsFile (
    const std::string & filename,
    const std::string & username,
    std::string & password )
```

Поиск пользователя в файле учетных записей

Аргументы

	in	filename	Имя файла с пользователями
	in	username	Логин для поиска
	out	password	Найденный пароль

Возвращает

true если пользователь найден, false если нет или ошибка

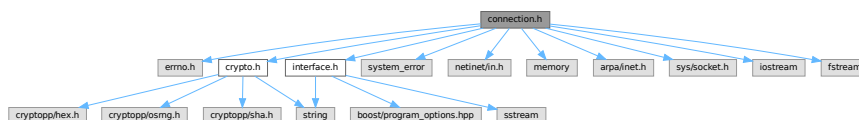
4.2 Файл connection.h

Заголовочный файл для класса [Connection](#) - установка сетевых соединений

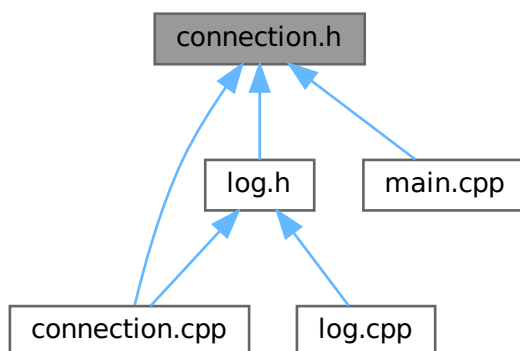
```
#include "errno.h"
#include "crypto.h"
#include "interface.h"
```

```
#include <system_error>
#include <netinet/in.h>
#include <memory>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <iostream>
#include <fstream>
```

Граф включаемых заголовочных файлов для connection.h:



Граф файлов, в которые включается этот файл:



Классы

- class [Connection](#)

Класс для управления сетевыми соединениями и аутентификацией

Макросы

- #define BUFFER_SIZE 1024

4.2.1 Подробное описание

Заголовочный файл для класса [Connection](#) - установка сетевых соединений

Автор

Мезин А.А.

Версия

1.0

Дата

2025

Авторство

ИБСТ ПГУ

Предупреждения

Учебный пример

4.3 connection.h

[См. документацию.](#)

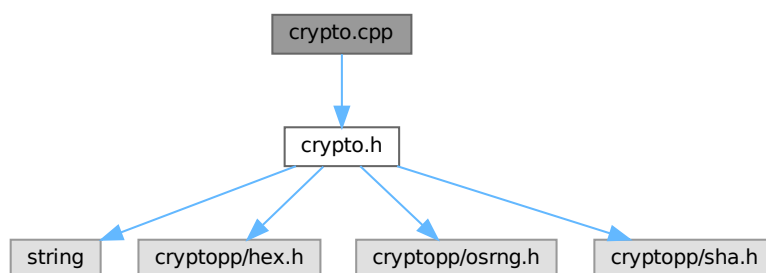
```
00001
00011 #pragma once
00012 #include "errno.h"
00013 #include "crypto.h"
00014 #include "interface.h"
00015 #include <system_error>
00016 #include <netinet/in.h>
00017 #include <memory>
00018 #include <arpa/inet.h>
00019 #include <sys/socket.h>
00020 #include <iostream>
00021 #include <fstream>
00022 #define BUFFER_SIZE 1024
00023 using namespace std;
00024
00031 class Connection{
00032 private:
00033
00034 public:
00042     static int establishConnection(const Params* p);
00043 };
```

4.4 Файл crypto.cpp

Реализация криптографических функций

```
#include "crypto.h"
```

Граф включаемых заголовочных файлов для crypto.cpp:



Функции

- string `auth` (string salt, string pass)
Вычисление хеша для аутентификации

4.4.1 Подробное описание

Реализация криптографических функций

Автор

Мезин А.А.

Версия

1.0

Дата

2025

Авторство

ИБСТ ПГУ

Предупреждения

Учебный пример

4.4.2 Функции

4.4.2.1 `auth()`

```
string auth (  
    string salt,  
    string pass )
```

Вычисление хеша для аутентификации

Функция аутентификации с использованием SHA-224.

Аргументы

	in	salt	Соль для усиления хеша
	in	pass	Пароль пользователя

Возвращает

Хеш SHA-224(salt + password) в hex-формате

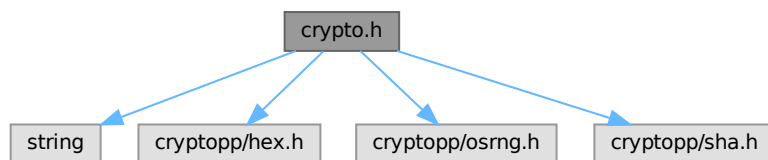
Использует библиотеку CryptoPP для вычисления хеша

4.5 Файл crypto.h

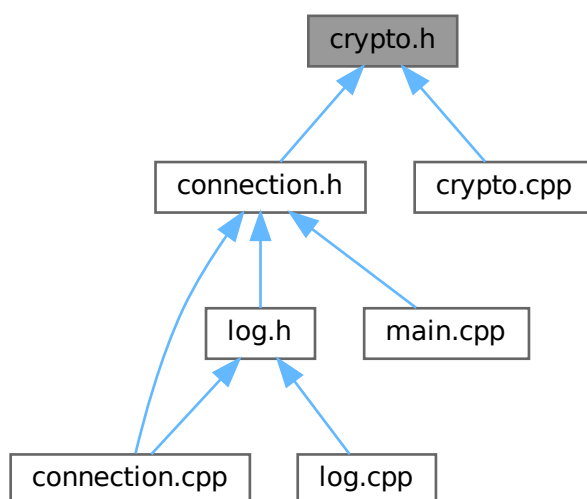
Заголовочный файл для криптографических функций

```
#include <string>
#include <cryptopp/hex.h>
#include <cryptopp/osrng.h>
#include <cryptopp/sha.h>
```

Граф включаемых заголовочных файлов для crypto.h:



Граф файлов, в которые включается этот файл:



Функции

- string `auth` (string salt, string pass)
Функция аутентификации с использованием SHA-224.

4.5.1 Подробное описание

Заголовочный файл для криптографических функций

Автор

Мезин А.А.

Версия

1.0

Дата

2025

Авторство

ИБСТ ПГУ

Предупреждения

Учебный пример

4.5.2 Функции

4.5.2.1 `auth()`

```
string auth (  
    string salt,  
    string pass )
```

Функция аутентификации с использованием SHA-224.

Аргументы

	in	salt	Соль для хеширования
	in	pass	Пароль для хеширования

Возвращает

Хеш-строка в шестнадцатеричном формате

Функция аутентификации с использованием SHA-224.

Аргументы

in	salt	Соль для усиления хеша
in	pass	Пароль пользователя

Возвращает

Хеш SHA-224(salt + password) в hex-формате

Использует библиотеку CryptoPP для вычисления хеша

4.6 crypto.h

[См. документацию.](#)

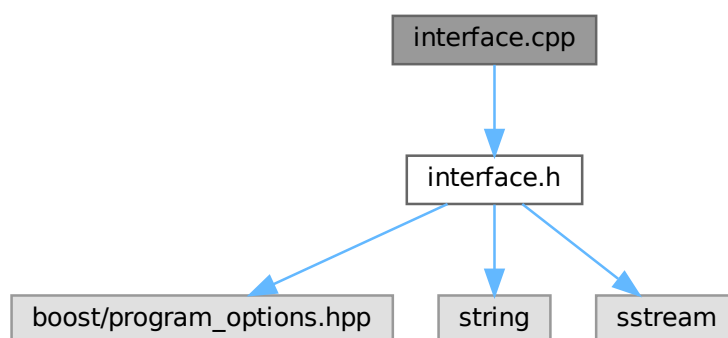
```
00001
00011 #pragma once
00012 #include <string>
00013 #include <cryptopp/hex.h>
00014 #include <cryptopp/osrng.h>
00015 #include <cryptopp/sha.h>
00016 using namespace std;
00017 namespace CPP = CryptoPP;
00018
00025 string auth(string salt, string pass);
```

4.7 Файл interface.cpp

Реализация пользовательского интерфейса

```
#include "interface.h"
```

Граф включаемых заголовочных файлов для interface.cpp:



4.7.1 Подробное описание

Реализация пользовательского интерфейса

Автор

Мезин А.А.

Версия

1.0

Дата

2025

Авторство

ИБСТ ПГУ

Предупреждения

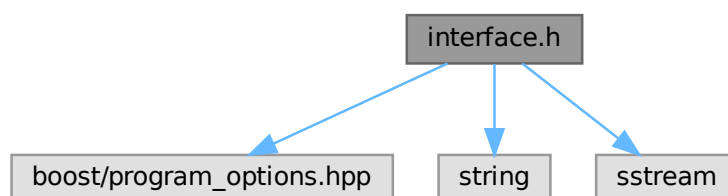
Учебный пример

4.8 Файл interface.h

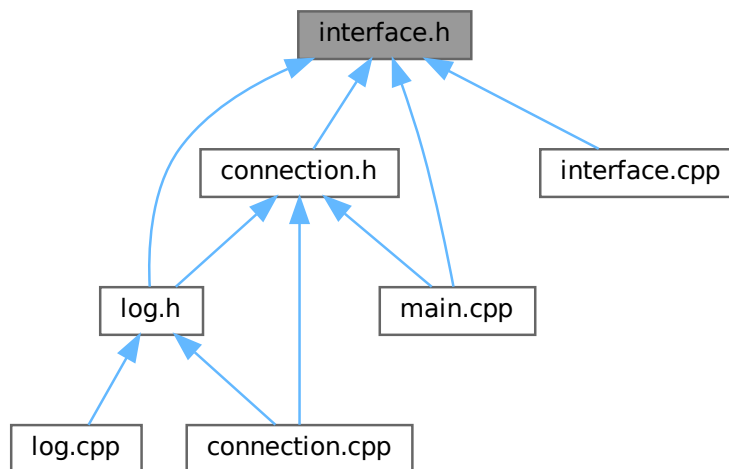
Заголовочный файл для пользовательского интерфейса

```
#include <boost/program_options.hpp>  
#include <string>  
#include <sstream>
```

Граф включаемых заголовочных файлов для interface.h:



Граф файлов, в которые включается этот файл:



Классы

- struct [Params](#)
Структура для хранения параметров командной строки
- class [UserInterface](#)
Класс для обработки параметров командной строки

4.8.1 Подробное описание

Заголовочный файл для пользовательского интерфейса

Автор

Мезин А.А.

Версия

1.0

Дата

2025

Авторство

ИБСТ ПГУ

Предупреждения

Учебный пример

4.9 interface.h

См. документацию.

```

00001
00011 #pragma once
00012 #include <boost/program_options.hpp>
00013 #include <string>
00014 #include <sstream>
00015 using namespace std;
00016 namespace po = boost::program_options;
00017
00022 struct Params {
00023     string inFileName;
00024     string inFileJournal;
00025     string inFileData;
00026     string logFile;
00027     int Port;
00028     string Address;
00029 };
00030
00036 class UserInterface {
00037 private:
00038     po::options_description desc;
00039     po::variables_map vm;
00040     Params params;
00041
00042 public:
00046     UserInterface();
00047
00054     bool Parser(int argc, const char** argv);
00055
00060     string getDescription();
00061
00066     Params getParams() {
00067         return params;
00068     };
00069 };

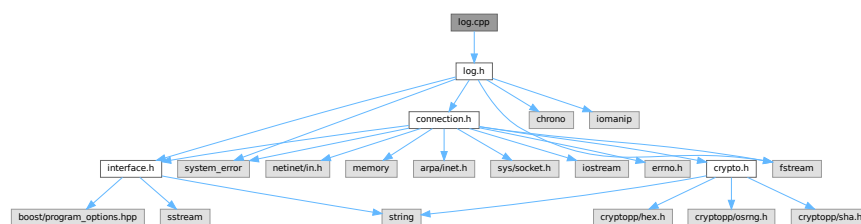
```

4.10 Файл log.cpp

Реализация функций логирования

```
#include "log.h"
```

Граф включаемых заголовочных файлов для log.cpp:



Функции

- `std::string getCurrentTime ()`
Получение текущего времени с миллисекундами
- `void logError (const std::string &logFile, const std::string &errorMessage)`
Запись ошибки в лог с временной меткой

4.10.1 Подробное описание

Реализация функций логирования

Автор

Мезин А.А.

Версия

1.0

Дата

2025

Авторство

ИБСТ ПГУ

Предупреждения

Учебный пример

4.10.2 Функции

4.10.2.1 `getCurrentTime()`

`std::string getCurrentTime ( )`

Получение текущего времени с миллисекундами

Получение текущего времени в формате строки

Возвращает

Строка в формате "ГГГГ-ММ-ДД ЧЧ:ММ:СС.ммм"

4.10.2.2 `logError()`

```
void logError (
    const std::string & logFile,
    const std::string & errorMessage )
```

Запись ошибки в лог с временной меткой

Запись ошибки в лог-файл

Аргументы

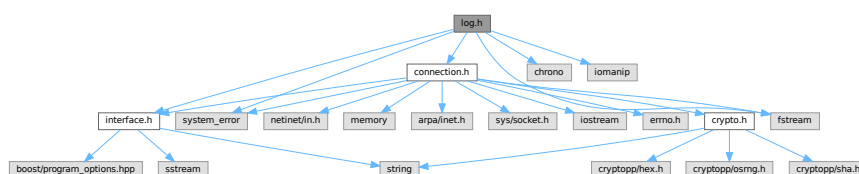
	in	logFile	Путь к файлу лога
	in	errorMessage	Текст ошибки для записи

4.11 Файл log.h

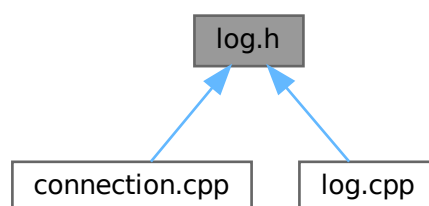
Заголовочный файл для функций логирования

```
#include "connection.h"
#include "interface.h"
#include <fstream>
#include <system_error>
#include <chrono>
#include <iomanip>
```

Граф включаемых заголовочных файлов для log.h:



Граф файлов, в которые включается этот файл:



Функции

- `std::string getCurrentTime()`
Получение текущего времени в формате строки
- `void logError(const std::string &logFile, const std::string &errorMessage)`
Запись ошибки в лог-файл

4.11.1 Подробное описание

Заголовочный файл для функций логирования

Автор

Мезин А.А.

Версия

1.0

Дата

2025

Авторство

ИБСТ ПГУ

Предупреждения

Учебный пример

4.11.2 Функции

4.11.2.1 `getCurrentTime()`

`std::string getCurrentTime ( )`

Получение текущего времени в формате строки

Возвращает

Строка с текущим временем в формате "ГГГГ-ММ-ДД ЧЧ:ММ:СС.ммм"

Получение текущего времени в формате строки

Возвращает

Строка в формате "ГГГГ-ММ-ДД ЧЧ:ММ:СС.ммм"

4.11.2.2 `logError()`

```
void logError (
    const std::string & logFile,
    const std::string & errorMessage )
```

Запись ошибки в лог-файл

Аргументы

	in	logFile	Имя файла лога
	in	errorMessage	Сообщение об ошибке

Запись ошибки в лог-файл

Аргументы

	in	logFile	Путь к файлу лога
	in	errorMessage	Текст ошибки для записи

4.12 log.h

[См. документацию.](#)

```

00001
00011 #include "connection.h"
00012 #include "interface.h"
00013 #include <fstream>
00014 #include <system_error>
00015 #include <chrono>
00016 #include <iomanip>
00017
00022 std::string getCurrentTime();
00023
00029 void logError(const std::string& logFile, const std::string& errorMessage);

```

4.13 Файл main.cpp

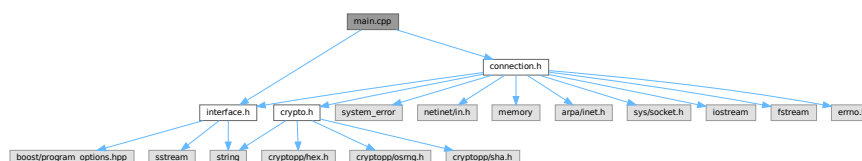
Главный модуль серверного приложения

```

#include "connection.h"
#include "interface.h"

```

Граф включаемых заголовочных файлов для main.cpp:



Функции

- int `main` (int argc, const char **argv)

Точка входа в приложение

4.13.1 Подробное описание

Главный модуль серверного приложения

Автор

Мезин А.А.

Версия

1.0

Дата

2025

Авторство

ИБСТ ПГУ

Предупреждения

Учебный пример

Основная функция приложения, обрабатывает аргументы командной строки и запускает сервер для обработки подключений

4.13.2 Функции

4.13.2.1 main()

```
int main (
    int argc,
    const char ** argv )
```

Точка входа в приложение

Аргументы

	in	argc	Количество аргументов
	in	argv	Массив аргументов

Возвращает

0 при успешном выполнении, 1 при ошибке

